

Draft certificate and review brief

PSEUDOSQUARE SEARCH / 14 SEPTEMBER 2026 / VERSION 1

DRAFT - INDEPENDENT REVIEW PENDING. This is a reproducibility and review packet, not a signed mathematical certificate or a completed independent audit of GPU arithmetic.

The completed Quad run reports five qualifying nonsquares in the interval $10^{25} < x \leq 2 \times 10^{25}$. Its smallest reported survivor is:

12299505549823064272744801

The CPU-only checker verifies this number through prime 389. Every worker reached its assigned endpoint. A separate streaming check of all copied batch records passes continuity, accounting, source/version lineage, and survivor checks.

Recorded search coverage	3 of 3 partitions complete
Committed batches	7,716,406
Wheel candidates screened	17,928,682,231,606,059
Passing perfect squares rejected	122,862,466,131
Distinct nonsquare survivors	5
Final worker completion	2026-09-14 13:18:11 UTC / 06:18:11 PDT

Claims submitted for review

C1, unconditional candidate bound: L_{379} , L_{383} , and L_{389} are each at most 12299505549823064272744801, based on independently checked candidate arithmetic.

C2, conditional minimum: $L_{379} = L_{383} = L_{389}$ = the number above, provided the reviewer accepts the enumeration/exclusion argument for this run and the published lower-bound computation through 10^{25} .

C3, conditional exclusion: $L_{397} > 2 \times 10^{25}$ under the same two dependencies, since all five reported survivors fail a test at or before prime 397.

Not established here: an independent full rerun, a formal verification of CUDA, priority as a new record, or reproduction of the historical search below 10^{25} . Authoring this packet did not launch another GPU search.

Candidate arithmetic and exact meaning

Definition used

For an odd prime p , L_p is the smallest positive nonsquare x satisfying $x \bmod 8 = 1$ and a nonzero quadratic residue modulo every odd prime $q \leq p$. Labels advance through primes: 379, 383, 389, 397. A qualifying number need not itself be prime.

Reported survivor (ascending)	Passes through	First failure
12299505549823064272744801	389	397
15650149438046685889225369	389	397
16097006495958886408753441	379	383
19836943388271112639203721	379	383
19849113888816140052681001	383	389

All five are within the searched interval, congruent to 1 modulo 8, and nonsquares. The offline verifier independently generates primes through 1000 and compares modular exponentiation (Euler criterion) with a binary Jacobi-symbol implementation. These are CPU checks independent of the production modules, not an independent enumeration of rejected numbers.

Reconstructible witness for the smallest survivor

$\text{floor}(\sqrt{x})$	3507065090616
Canonical t_p	790767687071310684
Canonical t_n	1723450262389676227
Reconstruction	$x = t_p M_n - t_n M_p$

The integer square-root check verifies $r^2 < x < (r+1)^2$. For each prime through 389, $\text{pow}(x, (q-1)/2, q)$ equals 1. At $q = 397$ it fails. Full witnesses for every survivor are in `verification.json`.

Why one minimum can cover three entries

Any 383- or 389-qualified number is also 379-qualified. If the smallest 379-qualified number is exhaustive and also passes 383 and 389, it is the minimum for all three labels. Conversely, a complete list through 379 can be filtered for 397; here none remains.

Extension-check trap: production `search.py` and `search_v2.py` define PRIMES only through 379. Calling their `qualifies(n, 389)` helper does not test 383 or 389. This packet uses a separate prime generator in `verify.py` for the higher-prime conclusions.

Coverage and accounting receipt

[illegible]

The checked t_p domain is [2230520, 2057046173387378757). The three adjacent partitions below cover this domain once at the ledger level. All final checkpoint cursors equal the listed end.

Worker o / COMPLETE

[2230520, 685682057797279932)

2,572,135 batches; 5,976,227,309,646,133 candidates; 40,954,079,504 rejected squares. Final checkpoint: 2026-09-14T11:58:12Z. Legacy/V2 committed batches: 62,058 / 2,510,077.

```
ledger-0.jsonl SHA-256
edbf6a2df08f982fb50725e3569fac8078e4922d03ff5991267e3043b53f633b
```

Worker 1 / COMPLETE

[685682057797279932, 1371364115592329344)

2,572,135 batches; 5,976,227,517,383,034 candidates; 40,954,171,702 rejected squares. Final checkpoint: 2026-09-14T11:25:46Z. Legacy/V2 committed batches: 63,291 / 2,508,844.

```
ledger-1.jsonl SHA-256
0e672f8fdb252e3e90608be0b63880d21bd038c2886c3b035312e2764abb404
```

Worker 2 / COMPLETE

[1371364115592329344, 2057046173387378757)

2,572,136 batches; 5,976,227,404,576,892 candidates; 40,954,214,925 rejected squares. Final checkpoint: 2026-09-14T13:18:11Z. Legacy/V2 committed batches: 9,735 / 2,562,401.

ledger-2.jsonl SHA-256
4a0ebf20c50498f6b46c3abaae665f1a6278a5d6b19fb6c226edb7747dc77b8d

The independent streaming scan found zero replay rows, gaps, partial trailing lines, or inconsistent totals in this frozen set. Each row satisfies $\text{passed} = \text{squares} + \text{number of reported nonsquare hits}$. Candidate totals count pairs reaching the match scan; they are not counts of integers that passed every prime test.

There are 122,862,466,136 post-prime-filter passes: 122,862,466,131 squares plus five nonsquares. The archive contains 4,390,182,234 uncompressed ledger bytes. Source hashes and ledger hashes identify bytes; they are not digital signatures or trusted timestamps.

Enumeration argument to inspect

This page is a proof sketch and review map, not a substitute for checking the frozen implementation. All arithmetic bounds refer to this specific campaign.

M_p	2057046173382917717
M_n	4483259527721526840
Factors of M_p	7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 53, 89
Factors of M_n	8, 3, 5, 47, 59, 61, 67, 71, 73, 79, 83, 97

1. Unique representation

The moduli are coprime. For each x choose $t_n = (-x M_p^{-1}) \bmod M_n$, in $[0, M_n)$. Then $t_p = (x + t_n M_p) / M_n$ is an integer and $x = t_p M_n - t_n M_p$. This canonical choice is unique.

2. Sufficient outer bounds

```
first = ceil(lo / M_n)
end = floor((hi - 1 + (M_n - 1) M_p) / M_n) + 1
```

For $lo \leq x < hi$ and canonical t_n , these bounds contain every possible t_p . Batches partition $[first, end)$ and split at M_p period boundaries. Reviewer: independently derive both floor/ceiling endpoints.

3. Correct residue sets

For q dividing M_p , require t_p in $M_n^{-1} \text{QR}(q)$. For q dividing M_n , require t_n in $(-M_p)^{-1} \text{QR}(q)$. Modulo 8, require $t_n = (-M_p)^{-1} \bmod 8$. QR excludes zero. Verify the inverse signs; using unscaled QR sets would be wrong.

4. Complete focus enumeration

Each admissible residue is built from CRT contributions. Rings are split into two groups C and D. For each d , binary searches on sorted C select sums in a requested window, with wrap = 0 and 1. Because c, d are in $[0, M)$, these two cases cover $(c+d) \bmod M$. Reviewer: check constants, empty groups, exact endpoints, period offsets, and injectivity.

Source map: search_v2.py:251, search_v2.py:113, search_v2.py:137, search_v2.py:87, search_v2.py:218 (all in sources/).

GPU exclusion and safety obligations

5. Every compatible pair reaches the predicate

The positive A window is sorted. With $\text{actual_a} = \min(A)$ and $\text{actual_b} = \max(A)+1$, the negative B bounds are tightened as follows. An empty A batch is committed with zero candidates.

```
bnlo = max(0, floor((actual_a M_n - hi) / M_p) + 1)
bnhi = min(M_n, floor(((actual_b-1) M_n - lo) / M_p) + 1)
```

For each t_n in B, the kernel binary-searches the first A product at least $t_n M_p + lo$, then scans while that product is below $t_n M_p + hi$. Review off-by-one cases, period splits, and empty windows independently.

6. Exact arithmetic and all prime filters

Products and differences use two 64-bit limbs. Both focus moduli are below 2^{63} , so adding two reduced CRT contributions fits in 64 bits. In this campaign t_p is below 2^{61} and t_n below 2^{62} ; their products and additions stay far below 2^{128} . Review carry, borrow, unsigned comparisons, and all intermediate bounds.

The focus primes cover odd primes through 97. The match filters cover every remaining odd prime 101 through 379, with zero residues forbidden. V2 reconstructs the wide x and reduces it for the first four filters; legacy uses packed scaled A residues plus scaled negative B residues. Both versions must be reviewed because both contributed committed coverage.

7. Square rejection and finite buffers

The square test uses a floating-point square-root estimate followed by integer correction and exact wide multiplication. Review termination and $r+1$ safety for $x \leq 2 \times 10^{25}$. Confirm this rejects only true squares.

A/B capacity is 67,108,864 entries. Over-capacity generation must return overflow and retry a smaller uncommitted window. The survivor buffer holds 100,000 values; excess must stop before coverage is committed. Review atomic counts, synchronization, bounds, and crash behavior. No full GPU rerun or fault-injection campaign was performed for this packet.

8. Memory claim and implementation scope

This engine stores large half-sum arrays: positive [66,718,080, 1,940,400] and negative [66,774,240, 1,912,680], plus batch buffers. Do not describe its total working memory as only the sum of admissible residue counts. The initial mixed-radix terms and the production working arrays are different quantities.

Source map: search_v2.py:258, search_v2.py:165, search_v2.py:181, search_v2.py:454. Public helper range: search_v2.py:27.

Version provenance and validation limits

This was a mixed-version run: 135,084 committed legacy batches and 7,581,322 V2 batches. Migration preserved the same numerical bounds, partitions, mathematical engine settings, and cumulative counters. All five hits occurred after migration.

Legacy search.py SHA-256
4ceca7948b26896603a039b482f6bad6823d4650357c1b47fa42d40c9150042e

Legacy generated match kernel SHA-256
64083cecaa903123b90264d92573c23381920a9fbf7f62c773e6b94223394bc2

V2 search_v2.py SHA-256
ed9a133c46b082051f8b47b399075a6f3ebe292d4694052bf131cf5bccccd4ab

V2 generated match kernel SHA-256
2ac7775b837330987996d5426c5b21b9b429872e10f732c85c2a1c1f097bad71

release-v2.json binds seven source files and three validation artifacts. Each worker references its migration record and the preserved pre-migration checkpoint. The full-ledger checker hashes each legacy prefix and verifies its boundary counters and every subsequent row identity.

Existing validation evidence, preserved unchanged

self-test.json / self-test-v2.json: direct small-range comparisons, singleton interval boundaries, high-value windows, recovery of published `L_373` with the full focus split, and rejection of the exact square 10000000000003925449108561. Generated focus values in a targeted window were checked for admissibility; this is not a full-focus cardinality/completeness proof.

replay-v2.json: one recorded production batch replayed with both implementations, 11 runs per implementation including two warmups, with matching A/B sizes, counts, squares, and hit lists. Nine timed samples each gave median 85.843 ms legacy and 50.161 ms V2. This is limited equivalence evidence, not independent full-search validation.

migration-tests-v2.json: ten tests covering locks, altered sources/scopes, tampered legacy evidence, preserved progress, crash tails, interrupted migration, versioned rows, and inconsistent duplicate rejection. These are historical test receipts.

Audit scope: production rollout.audit() checks ledger consistency, but its success label alone does not require all three workers to exist or be complete. verify.py explicitly checks the exact worker set, recomputed partition endpoints, COMPLETE status, and cursor=end. Neither checker validates every GPU rejection.

Offline reproduction and evidence inventory

Keep the small review packet and the separate ledgers.tar.gz archive together, with the archive next to the bundle directory. Python 3.10+ and its standard library suffice for the independent checks below. No network, CUDA, NumPy, or CuPy is needed.

Quick check: fingerprints, endpoints, and all survivors

```
cd bundle
python3 verify.py
```

Expected status: METADATA_AND_CANDIDATE_CHECKS_PASS. This does not read the full ledgers.

Full check: stream all recorded batches

```
python3 verify.py --ledgers ../ledgers.tar.gz
# To save a new receipt, use a new output filename:
python3 verify.py --ledgers ../ledgers.tar.gz \
--output /tmp/pseudosquare-review-result.json
```

Expected status: FULL_LEDGER_AND_CANDIDATE_CHECKS_PASS. The checker streams the compressed records without extracting 4.39 GB to disk. It checks the complete worker set, bounds, continuity, totals, no incomplete tail, row versions, migration prefixes, candidate arithmetic, and archive fingerprint. It fails closed on nonadjacent replays; none occur in this snapshot.

File or directory	Purpose
certificate.json	Machine-readable draft claims and unresolved dependencies
verification.json	New CPU-only full-ledger receipt and hit-batch locations
manifest.json	SHA-256 of packet files; excludes itself
sources/	Copied production sources; preserve unchanged
evidence/worker-*.json	Final complete checkpoints
evidence/lineage/	Three transitions and their before-checkpoints
evidence/release-v2.json	Pinned source and validation identities
evidence/*audit*.json	Historical rollout/resume audit receipts, not final receipts
../ledgers.tar.gz	All three full ledgers, 738,560,818 compressed bytes

verify.py checks the bundle manifest when present. The archive digest is recorded in verification.json and certificate.json. Use arbitrary-precision integer parsing: final totals exceed JavaScript's exact integer range. The verifier emits large computed totals as decimal strings.

Do not run migrate, operate_v2, search, or replay_v2 against production or frozen evidence. replay_v2 writes its evidence file. Any GPU validation must use a fresh scratch directory and an explicitly available GPU; validation/replays are not novel discovery coverage.

Independent reviewer instructions and verdict

Reviewer task: try to falsify the completeness claim. Do not accept the candidate values, hashes, or a PASS label as proof that all smaller numbers were examined.

R1 - Evidence: independently run both checker modes, compare archive and ledger digests, verify the exact three partitions and all transition records. Review `verify.py` itself. Explain the limits of an unsigned, author-assembled packet.

R2 - Representation: derive the canonical two-focus representation and both outer bounds. Confirm all relevant x map to an included pair exactly once.

R3 - Generation: inspect scaled residue sets, CRT contributions, ring partitioning, wrap intervals, sorting/binary searches, and B-window tightening. Look for admissible values omitted at boundaries.

R4 - Arithmetic: inspect legacy and V2 modular tests, wide arithmetic, square correction, atomic counters, capacity behavior, and synchronization. The legacy prefix cannot be justified solely by V2 tests.

R5 - Persistence: verify flush-before-checkpoint ordering, overflow retry semantics, interrupted-batch handling, and version-pinned continuation. Distinguish committed coverage from completed process execution.

R6 - External dependency: verify the definition and $L_{379} > 10^{25}$ in Sorenson, Section 2 (PDF page 2), and state explicitly whether you accept that published computation. It was not rerun here.

R7 - Consequences: recompute all candidate tests through at least 397 without the production PRIMES cap. Check the nested-set reasoning for equality at 379/383/389 and the conditional lower bound at 397.

R8 - Further validation: specify targeted independent boundary tests or a separately implemented full rerun if needed. Do not label another run of the identical implementation an independent algorithmic verification. Seek authorization before assigning additional production compute.

Reviewer identity / model: PENDING

Reviewed source and archive digests: PENDING

Verdict: PENDING - accept conditional claim / require changes / reject

Blocking findings and required checks: PENDING

Required review deliverable

Return a separate review document with a verdict for C1, C2, and C3; file:line references for each finding; commands and test results; accepted external assumptions; and remaining risks. Preserve this frozen packet and record any proposed patches separately. No reviewer has signed off on C2 or C3 in this draft.

External reference and data origin

Jonathan P. Sorenson, arXiv:1001.3316v2 (22 January 2010), *Sieving for pseudosquares and pseudocubes in parallel using doubly-focused enumeration and wheel datastructures*, Section 2. Used here for the definition and historical lower bound only.

Primary run evidence: Quad, `/home/alex/erdos/overnight/pseudosquare-379-2026-09-12`. Source: `/home/alex/erdos/research/pseudosquare-search`. The snapshot was copied after all worker endpoints were complete; full ledger bytes were hashed locally and compared with the originals.